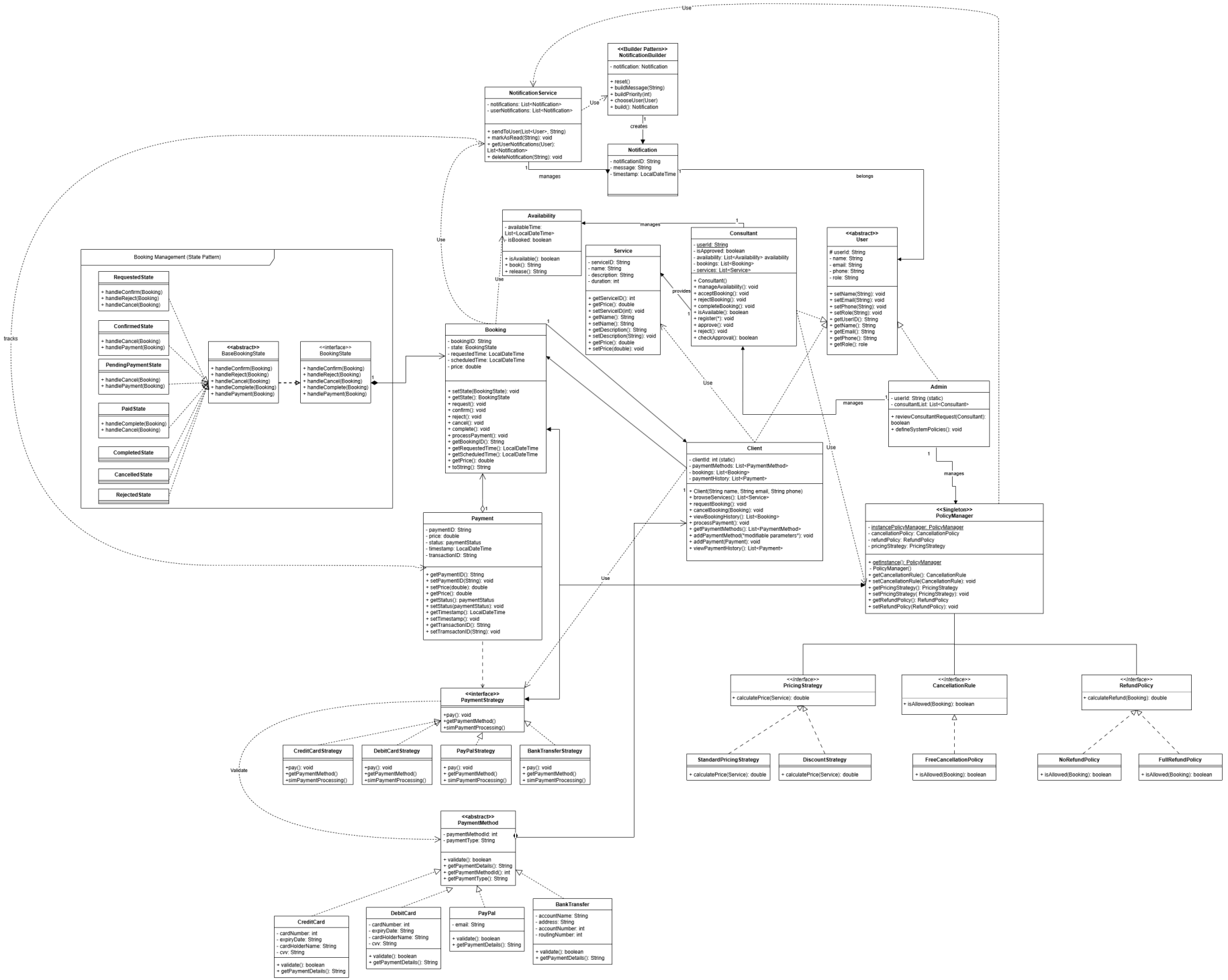
Use
<<Builder Pattern>>
NotificationBuilder
- notification: Notification
+ reset()
+ buildMessage(String)
+ buildPriority(int)
+ chooseUser(User)
+ build(): Notification
Use
Notification Service
- notifications: List<Notification>
- userNotifications: List<Notification>
+ sendToUser(List<User>, String)
+ markAsRead(String): void
+ getUserNotifications(User): List<Notification>
+ deleteNotification(String): void
1
creates
Notification
- notificationID: String
- message: String
- timestamp: LocalDateTime
1
manages
1
belongs
Availability
- availableTime: List<LocalDateTime>
- isBooked: boolean
+ isAvailable(): boolean
+ book(): String
+ release(): String
manages
1
Consultant
- userId: String
- isApproved: boolean
- availability: List<Availability> availability
- bookings: List<Booking>
- services: List<Service>
+ Consultant()
+ manageAvailability(): void
+ acceptBooking(): void
+ rejectBooking(): void
+ completeBooking(): void
+ isAvailable(): boolean
+ register(*): void
+ approve(): void
+ reject(): void
+ checkApproval(): boolean
<<abstract>>
User
userId: String
- name: String
- email: String
- phone: String
- role: String
+ setName(String): void
+ setEmail(String): void
+ setPhone(String): void
+ setRole(String): void
+ getUserID(): String
+ getName(): String
+ getEmail(): String
+ getPhone(): String
+ getRole(): role
Use
Service
- serviceID: String
- name: String
- description: String
- duration: int
+ getServiceID(): int
+ getPrice(): double
+ setServiceID(int): void
+ getName(): String
+ setName(): String
+ getDescription(): String
+ setDescription(String): void
+ getPrice(): double
+ setPrice(double): void
provides
1
Booking Management (State Pattern)
RequestedState
+ handleConfirm(Booking)
+ handleReject(Booking)
+ handleCancel(Booking)
ConfirmedState
+ handleCancel(Booking)
+ handlePayment(Booking)
PendingPaymentState
+ handleCancel(Booking)
+ handlePayment(Booking)
PaidState
+ handleComplete(Booking)
+ handleCancel(Booking)
CompletedState
CancelledState
RejectedState
tracks
<<abstract>>
BaseBookingState
+ handleConfirm(Booking)
+ handleReject(Booking)
+ handleCancel(Booking)
+ handleComplete(Booking)
+ handlePayment(Booking)
<<interface>>
BookingState
+ handleConfirm(Booking)
+ handleReject(Booking)
+ handleCancel(Booking)
+ handleComplete(Booking)
+ handlePayment(Booking)
1
Booking
- bookingID: String
- state: BookingState
- requestedTime: LocalDateTime
- scheduledTime: LocalDateTime
- price: double
+ setState(BookingState): void
+ getState(): BookingState
+ request(): void
+ confirm(): void
+ reject(): void
+ cancel(): void
+ complete(): void
+ processPayment(): void
+ getBookingID(): String
+ getRequestedTime(): LocalDateTime
+ getScheduledTime(): LocalDateTime
+ getPrice(): double
+ toString(): String
1
Use
Admin
- userId: String (static)
- consultantList: List<Consultant>
+ reviewConsultantRequest(Consultant): boolean
+ defineSystemPolicies(): void
manages
1
manages
Client
- clientId: int (static)
- paymentMethods: List<PaymentMethod>
- bookings: List<Booking>
- paymentHistory: List<Payment>
+ Client(String name, String email, String phone)
+ browseServices(): List<Service>
+ requestBooking(): void
+ cancelBooking(Booking): void
+ viewBookingHistory(): List<Booking>
+ processPayment(): void
+ getPaymentMethods(): List<PaymentMethod>
+ addPaymentMethod(*modifiable parameters*): void
+ addPayment(Payment): void
+ viewPaymentHistory(): List<Payment>
1
Use
<<Singleton>>
PolicyManager
- instancePolicyManager: PolicyManager
- cancellationPolicy: CancellationPolicy
- refundPolicy: RefundPolicy
- pricingStrategy: PricingStrategy
+ getInstance(): PolicyManager
- PolicyManager()
+ getCancellationRule(): CancellationRule
+ setCancellationRule(CancellationRule): void
+ getPricingStrategy(): PricingStrategy
+ setPricingStrategy(PricingStrategy): void
+ getRefundPolicy(): RefundPolicy
+ setRefundPolicy(RefundPolicy): void
Payment
- paymentID: String
- price: double
- status: paymentStatus
- timestamp: LocalDateTime
- transactionID: String
+ getPaymentID(): String
+ setPaymentID(String): void
+ setPrice(double): double
+ getPrice(): double
+ getStatus(): paymentStatus
+ setStatus(paymentStatus): void
+ getTimestamp(): LocalDateTime
+ setTimestamp(): void
+ getTransactionID(): String
+ setTramsactonID(String): void
Use
<<interface>>
PaymentStrategy
+pay(): void
+getPaymentMethod()
+simPaymentProcessing()
CreditCardStrategy
+pay(): void
+getPaymentMethod()
+simPaymentProcessing()
DebitCardStrategy
+pay(): void
+getPaymentMethod()
+simPaymentProcessing()
PayPalStrategy
+ pay(): void
+ getPaymentMethod()
+ simPaymentProcessing()
BankTransferStrategy
+ pay(): void
+ getPaymentMethod()
+ simPaymentProcessing()
Validate
<<abstract>>
PaymentMethod
- paymentMethodId: int
- paymentType: String
+ validate(): boolean
+ getPaymentDetails(): String
+ getPaymentMethodId(): int
+ getPaymentType(): String
CreditCard
- cardNumber: int
- expiryDate: String
- cardHolderName: String
- cvv: String
+ validate(): boolean
+ getPaymentDetails(): String
DebitCard
- cardNumber: int
- expiryDate: String
- cardHolderName: String
- cvv: String
+ validate(): boolean
+ getPaymentDetails(): String
PayPal
- email: String
+ validate(): boolean
+ getPaymentDetails(): String
BankTransfer
- accountName: String
- address: String
- accountNumber: int
- routingNumber: int
+ validate(): boolean
+ getPaymentDetails(): String
<<Interface>>
PricingStrategy
+ calculatePrice(Service): double
<<Interface>>
CancellationRule
+ isAllowed(Booking): boolean
<<Interface>>
RefundPolicy
+ calculateRefund(Booking): double
StandardPricingStrategy
+ calculatePrice(Service): double
DiscountStrategy
+ calculatePrice(Service): double
FreeCancellationPolicy
+ isAllowed(Booking): boolean
NoRefundPolicy
+ isAllowed(Booking): boolean
FullRefundPolicy
+ isAllowed(Booking): boolean